# MONETO




Realización: David Cabrera Briales

Supervisión: Olga Murillo Colino

Entrega: 30 de mayo de 2025

# Contenidos

1. Introducción ..... 3

2. Estado del arte ..... 6

3. Metodología ..... 11

4. Tecnologías y herramientas ..... 14

5. Planificación, Diagnóstico y Contexto Laboral ..... 18

6. Análisis ..... 22

7. Diseño ..... 31

8. Ejecutable y pruebas ..... 42

9. Conclusiones ..... 46

10. Vías futuras ..... 47

11. Bibliografía y Webgrafía ..... 50

Anexos ..... 52

# 1. Introducción

Después de haber reflexionado durante unos cuantos días, de haber investigado en el mercado de las aplicaciones y de haber realizado un estudio de las apps más y menos usadas, así como de lo que realmente resulta más útil en el día a día de una persona, se llegó a la conclusión de que existe una aplicación con potencial para generar un gran impacto en el mercado actual.

Fue tomada la decisión de desarrollar esta aplicación, ya que se considera que una de las mayores preocupaciones del ser humano actual es el control de sus finanzas: conocer la cantidad de dinero ingresado en un mes determinado, saber cuánto ha sido gastado en ese mismo mes y en qué se ha realizado dicho gasto, ya sea en el hogar, en el vehículo, entre otros.

La elección de este tipo de aplicación en lugar de otra fue motivada por la dificultad personal que se experimenta al gestionar ingresos, gastos y finanzas de forma ordenada y escalonada. No se solía anotar ni el dinero recibido ni en qué se gastaba. Al considerar ambos aspectos, fue vista como una buena oportunidad la posibilidad de ayudar a otras personas a resolver el problema de no saber cómo gestionar sus finanzas adecuadamente.

## 1.1. Objetivos de la aplicación:

### 1.1.1. Objetivos generales

- Tener un control de los ingresos
- Tener un control de los gastos
- Ayudarte en el control de tus finanzas

### 1.1.2.Objetivos específicos

- Tener un control de los gastos de transporte
  - Gastos del coche, tales como revisión, mantenimientos y gasto en repostaje
  - Gastos de transporte público, ya sea en, taxi, bus o metro/tren
  - Gastos en viajes, tanto en barco como en avión, ambos los métodos de transporte más comunes a la hora de viajar, y ahí mismo también puedes anotar los gastos del propio viaje, tales como, alojamiento, comidas y otros tipos de gastos, como, por ejemplo, excursiones o actividades rurales
- Control de gastos de tus hobbies favoritos
  - Te hará un pequeño cuestionario acerca de tus hobbies favoritos, en este caso, se te muestra una lista con los hobbies más comunes, puedes seleccionar tantos como quieras
  - Cuando ya tengas los hobbies seleccionados, podrás ingresas en cada categoría el dinero gastado, la fecha en la que hiciste ese gasto, y en que gastaste
- Control de gastos del hogar
  - Gastos del alquiler
  - Gasto del agua
  - Gasto de luz
  - Gasto de internet
  - Gasto de compras
- Control de ahorros
  - Dentro de la aplicación, hay un apartado ahorros, lo cual actúa como una hucha virtual, en la que puedes ir agregando dinero
- Control de ingresos
  - Apartado de la aplicación, en el cual puedes seleccionar mes y año, y agregar la nómina de dicho mes y año

- Control general o reportes
  - Es un apartado, en el cual, puedes o bien descargar un PDF y veras un documento detallado con tus gastos o un gráfico comparando gastos e ingresos
  - Si lo que quieres es un documento detallado con todo, gastos, ingresos y ahorros, lo tienes al alcance con un solo clic a un botón
  - O si lo único que quieres es una ligera idea de lo ingresado o gastado, hay un botón, que sin descargar nada, en la propia aplicación te muestra un gráfico comparando ambas cosas, para asi hacerte una idea de tus finanzas

## 1.2 Abstract

This Final Degree Project focuses on the development of a mobile application designed to facilitate personal finance management for users of all backgrounds. The main objective is to offer an intuitive and user-friendly tool that allows individuals to keep track of their monthly income and expenses in an organized way. Users can classify their spending into key categories such as transport, housing, and leisure, while also setting aside money in a virtual savings section. The app includes a feature to generate a downloadable PDF report containing detailed financial records, which helps users analyze their habits. Additionally, a simple graphical interface provides a visual comparison between the total income and expenses for each month, encouraging better decision-making and promoting financial awareness

# 2. Estado del arte

## 2.1 Investigación

La gestión de las finanzas personales, hoy en día, se ha convertido en una necesidad creciente en una sociedad marcada principalmente por el consumo y la digitalización. La falta de educación financiera provoca un aumento del gasto impulsivo, lo que ha motivado a los programadores y desarrolladores de software a desarrollar herramientas orientadas a facilitar el control económico individual.

### 2.1.1 Revisión de aplicaciones existentes

Para este apartado, se han elegido cuatro aplicaciones existentes en el mercado, las cuales son:

- Fintonic
- Spendee
- Mint
- Money Manager

### 2.1.2 Funcionalidades

- Fintonic:
  - Permite agregar cuentas bancarias
  - Análisis de gastos e ingresos
- Spendee:
  - Versión web
  - Presupuestos inteligentes
- Mint:
  - Seguimiento carteras de inversión
  - Información del mercado

- Money Manager:
  - Creación de presupuestos
  - Recordatorio

### 2.1.3 Experiencia de usuario

- Fintonic:
  - Buen soporte al cliente
  - Interfaz intuitiva
- Spendee:
  - Clara visualización de datos
  - Buena sincronización con cuentas bancarias
- Mint:
  - Buena relación calidad-precio
  - Fácil uso
- Money Manager:
  - Buena generación de reportes
  - Interfaz intuitiva

### 2.1.4 Puntos débiles

- Fintonic:
  - Seguridad de los datos, inseguridad a la hora de conectar la cuenta bancaria
  - Intereses elevados en ciertos prestamos
- Spendee:
  - Errores con los cálculos
  - Ciertos problemas de seguridad
- Mint:
  - Fallos repetitivos que hacen que la app se bloquee con frecuencia
  - Mala atención al cliente

- Money Manager:
  - Cierres inesperados
  - Limitaciones en la versión gratuita, no es una app 100% gratuita, funcionalidades avanzadas requieren de una cuota bastante alta

### 2.1.5 Tabla comparativa

| | *Puntos fuertes* | *Puntos débiles* |
|---|---|---|
| *Fintonic* | Interfaz clara e intuitiva | Mala seguridad de los datos |
| | Análisis de gastos e ingresos | Intereses elevados |
| *Spendee* | Consta con versión web | Errores con los cálculos |
| | Visualización clara de los datos | Brechas de seguridad con la versión web |
| *Mint* | Fácil uso | Es totalmente de pago |
| | Información del mercado | La aplicación se bloquea con frecuencia |
| *Money manager* | Interfaz intuitiva | Cierres inesperados |
| | Recordatorios de pago | Funcionalidades avanzadas son de pago |

## 2.2. Análisis

### 2.2.1 Tecnologías usadas

- Fintonic:
  - Frontend: React
  - Backend: Java y Python como lenguajes de programación. Spring como framework, MySQL y Postgre SQL como bases de datos. Lambda como servicio de almacenamiento, Stripe como servicio de

pago. Burp Suite como herramienta de seguridad y para la gestión de dependencias utiliza Maven y Pip.

- Spendee:
  - Frontend: HTML, CSS y JavaScript para su versión web, y React, Angular y Vue.js para su versión de escritorio
  - Backend: Python, Java, PHP, JavaScript con node.js y Ruby como lenguajes de programación. Django, Ruby on Rails, Laravel y Spring como frameworks. MySQL, PostgreSQL, MongoDB, Firebase y MariaDB como bases de datos.
- Mint:
  - Frontend: React, angular y vue.js
  - Backend: Python, Java y C# como lenguajes de programación. Django, Laravel, Springs y Ruby on Rails como frameworks. MySQL o NoSQL como bases de datos relacionales.
- Money Manager:
  - Frontend: React, angular y vue.js
  - Backend: Python y java como lenguajes de programación. Django, Spring o Ruby on Rails como frameworks. MySQL y PostgreSQL como bases de datos.

## 2.3 Análisis crítico y posicionamiento de mi proyecto

### 2.3.1 ¿Qué problema aborda mi aplicación?

El problema principal que pretende abordar esta aplicación es la **mala gestión de los gastos personales**, una situación común que afecta a muchas personas debido a la falta de herramientas accesibles, simples y comprensibles. Muchas soluciones actuales en el mercado presentan interfaces complejas, funciones innecesarias o requieren el acceso a datos bancarios sensibles.

Esta aplicación propone una alternativa **100% gratuita, estable y fácil de usar**, centrada en ofrecer al usuario un control claro sobre su dinero, sin complicaciones ni barreras técnicas. Su diseño intuitivo y funcional está orientado a quienes desean

simplemente **visualizar cuánto ingresan y en qué gastan**, sin necesidad de crear presupuestos, analizar inversiones o conectarse a internet constantemente.

### 2.3.2 Aportaciones del proyecto frente a soluciones existentes

- Interfaz clara y sencilla:
  - La aplicación prioriza la simplicidad y la usabilidad, permitiendo que cualquier usuario, sin conocimientos financieros, pueda manejarla sin complicaciones
- Funcionamiento sin conexión a internet:
  - A diferencia de otras apps que requieren conexión constante para sincronizar datos bancarios, esta aplicación permite registrar gastos e ingresos de forma totalmente offline.
- Estabilidad de la aplicación:
  - La app está optimizada para evitar cierres inesperados o errores frecuentes, ofreciendo una experiencia robusta y fluida incluso en dispositivos de gama media o baja.
- Sin necesidad de vincular cuentas bancarias ni aportar datos sensibles:
  - Solo se requiere un correo electrónico y contraseña para acceder, garantizando mayor privacidad al no manejar datos financieros reales o personales.
- Visualización clara de los movimientos:
  - Los registros se muestran con fecha en formato dd/mm/yyyy, junto a la cantidad ingresada y una descripción proporcionada por el usuario. Este formato permite un control rápido e intuitivo.
- Enfoque minimalista y directo:
  - A diferencia de otras apps que ofrecen múltiples funciones (presupuestos, inversiones, mercado), esta se enfoca exclusivamente en mostrar al usuario en qué gasta, cuánto gasta y cuánto ingresa. Ideal para usuarios que solo quieren **control básico y efectivo de sus finanzas**.

# 3. Metodología

## 3.1. Modelo de desarrollo

Para el desarrollo de esta aplicación se ha optado por un enfoque ágil, concretamente inspirado en la metodología **Scrum**, aunque adaptado al trabajo individual. Esta decisión responde a la necesidad de gestionar el tiempo de manera flexible, dividiendo el proyecto en fases manejables que permitieran iterar sobre el producto e ir mejorándolo progresivamente.

La metodología se implementó mediante **sprints informales de una a dos semanas**, permitiendo centrar cada bloque de trabajo en una funcionalidad concreta (interfaz, base de datos, lógica de control de gastos, generación de informes, etc.). Esta organización facilitó una evolución progresiva del desarrollo y la detección temprana de posibles errores o mejoras.

## 3.2. Ciclo de vida del proyecto

A continuación, se detallan las fases principales seguidas durante el desarrollo del proyecto:

### 3.2.1.Planificación

En esta primera fase se definió el objetivo general del proyecto, el público al que va dirigido y los requerimientos funcionales mínimos. Se estructuró un calendario estimado con bloques de trabajo, y se seleccionaron las herramientas necesarias para el desarrollo:

- **Lenguaje y entorno:** Flutter (Dart) para desarrollo móvil.
- **Diseño visual:** Estilo inspirado en componentes de Jetpack Compose.
- **Almacenamiento local:** SQLite para los datos financieros y SharedPreferences para la gestión básica de usuarios.

### 3.2.2.Análisis de requisitos

En esta etapa se determinaron los **requisitos funcionales** y **no funcionales** de la aplicación.

- **Requisitos funcionales:**

  - Registro de ingresos y gastos con fechas, cantidad y descripción
  - Visualización clara y ordenada de los movimientos
  - Generación de informes en formato PDF
  - Acceso mediante un sistema básico de autenticación (correo y contraseña)

- **Requisitos no funcionales:**

  - Interfaz intuitiva y adaptada a todo tipo de usuarios
  - Funcionamiento sin necesidad de una constante conexión a internet
  - Aplicación estable, sin cierres inesperados
  - Privacidad: no se solicitan datos bancarios ni personales sensibles

También se definió el público objetivo: usuarios que buscan una herramienta sencilla para llevar un control básico de sus finanzas personales sin complicaciones.

### 3.2.3.Diseño

Durante esta fase se diseñó la estructura de la aplicación, tanto a nivel visual como lógico.

- A nivel visual, se creó un esquema de interfaz limpia, con pantallas simples y navegación directa. se utilizó la tecnología Flutter con componentes de diseño inspirados en **Jetpack Compose**, manteniendo una estética moderna pero simple.

- A nivel de arquitectura, se estableció una separación clara entre la lógica de presentación, la lógica de negocio y el acceso a datos, garantizando un desarrollo ordenado y fácilmente mantenible.

### 3.2.4.Implementación

La implementación se realizó en módulos progresivos, siguiendo la planificación establecida:

- Desarrollo de pantallas de registro, login y pantalla principal.
- Implementación de la lógica de almacenamiento de ingresos y gastos mediante **SQLite**.
- Uso de **SharedPreferences** para almacenar la información de usuario de forma segura y local.
- Desarrollo del generador de informes PDF.
- Añadido de una sección gráfica para visualizar el balance mensual de forma visual.

### 3.2.5.Pruebas

Durante la fase de pruebas se realizaron las siguientes acciones:

- **Pruebas unitarias**: para verificar el funcionamiento correcto de funciones individuales como cálculos, inserción de datos, exportación, etc.
- **Pruebas funcionales**: simulando el uso completo de la aplicación, desde el registro hasta la exportación de informes.
- **Pruebas de usabilidad**: con usuarios reales (entorno cercano) para comprobar la experiencia de uso, simplicidad de la interfaz y posibles mejoras.

- **Pruebas de estabilidad**: especialmente en situaciones sin conexión, con múltiples datos ingresados o cierres forzados.

## 3.3. Conclusión de la metodología

El enfoque ágil aplicado ha permitido adaptar el desarrollo a las necesidades reales del proyecto, facilitando correcciones rápidas y mejoras continuas. Gracias a la elección de herramientas ligeras y tecnologías como Flutter, SQLite y SharedPreferences, se ha logrado crear una aplicación funcional, estable y adecuada al perfil de usuario objetivo.

# 4. Tecnologías y herramientas

En esta sección se describen las tecnologías, frameworks, bibliotecas y herramientas utilizadas en el desarrollo del proyecto. La elección de cada una de ellas ha estado basada en criterios de compatibilidad, facilidad de uso, eficiencia y adecuación a los objetivos del proyecto: construir una aplicación móvil multiplataforma, ligera, intuitiva y totalmente funcional sin necesidad de conexión a internet.

## 4.1. Lenguajes de programación y framework

**Flutter (Dart)**

Para el desarrollo de la aplicación se utilizó **Flutter**, un framework de código abierto desarrollado por Google, que permite crear aplicaciones nativas para Android e iOS a partir de una única base de código escrita en **Dart**.

La elección de Flutter se justifica por varias razones:

- **Multiplataforma**: Permite desarrollar simultáneamente para Android e iOS, lo que ahorra tiempo y recursos.

- **Alto rendimiento**: Compila a código nativo, lo que proporciona una experiencia fluida.

- **Interfaz moderna**: Flutter incluye un conjunto amplio de widgets personalizables que permiten crear interfaces atractivas y funcionales de forma rápida.

- **Amplia comunidad y documentación**: Existen numerosos recursos, paquetes y ejemplos disponibles, lo que facilita el desarrollo y la resolución de problemas.

## 4.2. Entorno de desarrollo

**Android Studio**

El entorno de desarrollo empleado fue **Android Studio**, la IDE oficial para el desarrollo de aplicaciones Android, aunque completamente compatible con Flutter mediante el plugin correspondiente. Esta herramienta ofrece:

- Asistencia inteligente en la escritura de código (autocompletado, sugerencias, refactorización).

- Emuladores integrados para pruebas rápidas.

- Integración con Git para control de versiones.

- Herramientas de depuración, análisis de rendimiento y pruebas.

La elección de Android Studio responde a su estabilidad, integración directa con Flutter y herramientas completas para desarrollo móvil.

## 4.3. Base de datos

**SQLite**

Para la persistencia de datos financieros (ingresos, gastos y descripciones), se optó por utilizar **SQLite**, una base de datos relacional embebida, ampliamente utilizada en entornos móviles.

Justificación de su elección:

- Permite el almacenamiento local de datos sin necesidad de conexión a internet.
- Ligera, rápida y con bajo consumo de recursos.
- Bien soportada por Flutter a través de paquetes como sqflite.

**Paquete utilizado: sqflite: ^2.0.0+4**

Este paquete permite realizar operaciones CRUD (Create, Read, Update, Delete) sobre bases de datos SQLite desde Flutter de forma sencilla, mediante funciones asincrónicas.

## 4.4. Almacenamiento local

**SharedPreferences**

Se utilizó la librería **shared_preferences: ^2.2.2** para guardar información básica del usuario (como su correo, usuario y contraseña) sin necesidad de configurar una base de datos más compleja para ello. Este sistema de almacenamiento clave-valor es ideal para configuraciones simples.

## 4.5. Librerías y paquetes adicionales

A lo largo del desarrollo se integraron distintas bibliotecas de Flutter para cubrir funcionalidades especificas

| *Librería* | *Uso principal* |
|---|---|
| *Path y path provider* | Para obtener rutas específicas del sistema de archivos |
| *Pdf y printing* | Generación de informes formato PDF y opción de impresión directa |
| *intl* | Formateo de fechas, monedas y tiempos, adaptados a la localización española |
| *Fl_chart* | Representación gráfica de ingresos y gastos mensuales de forma sencilla y visual |

Cada una de estas bibliotecas fue seleccionada por su compatibilidad con Flutter, su buena documentación y su rendimiento estable en entornos móviles.

## 4.6. Control de versiones

Para el control de versiones del proyecto se utilizó **Git**, integrado directamente en Android Studio. Git permitió llevar un seguimiento claro de los avances, probar funcionalidades nuevas sin afectar el código principal y realizar copias de seguridad locales durante el desarrollo.

## 4.7. Compatibilidad multiplataforma

Gracias al uso de Flutter, la aplicación fue desarrollada pensando en su posible publicación tanto en **Google Play Store (Android)** como en la **App Store (iOS)**. Aunque en esta fase se ha centrado principalmente en pruebas para Android, la arquitectura utilizada permite escalar fácilmente el proyecto a dispositivos Apple sin cambios significativos en el código.

## 4.8. Conclusión

La elección de estas tecnologías ha permitido desarrollar una aplicación robusta, intuitiva y funcional, alineada con los objetivos del proyecto. Flutter, combinado con SQLite y

SharedPreferences, ha ofrecido una solución eficiente y completamente operativa sin depender de conexiones externas, lo cual resulta esencial en una aplicación orientada a la gestión personal de finanzas.

# 5. Planificación, Diagnóstico y Contexto Laboral

## 5.1. Diagnóstico y contexto

En un contexto marcado por la digitalización y el auge del consumo, muchas personas continúan enfrentando dificultades a la hora de gestionar sus finanzas personales. La falta de educación financiera, el uso excesivo de métodos de pago digitales y la escasa organización del presupuesto doméstico hacen que muchos usuarios pierdan el control sobre sus gastos, llegando incluso a situaciones de endeudamiento innecesario.

El proyecto surge con el propósito de ofrecer una solución sencilla y funcional para ayudar a estas personas a **tomar conciencia de sus hábitos financieros**. A diferencia de muchas aplicaciones complejas o con modelos de pago, esta aplicación busca ser **ligera, gratuita y sin complicaciones**, ofreciendo funciones básicas pero esenciales: registrar ingresos y gastos, visualizar un balance mensual, y exportar un resumen en PDF.

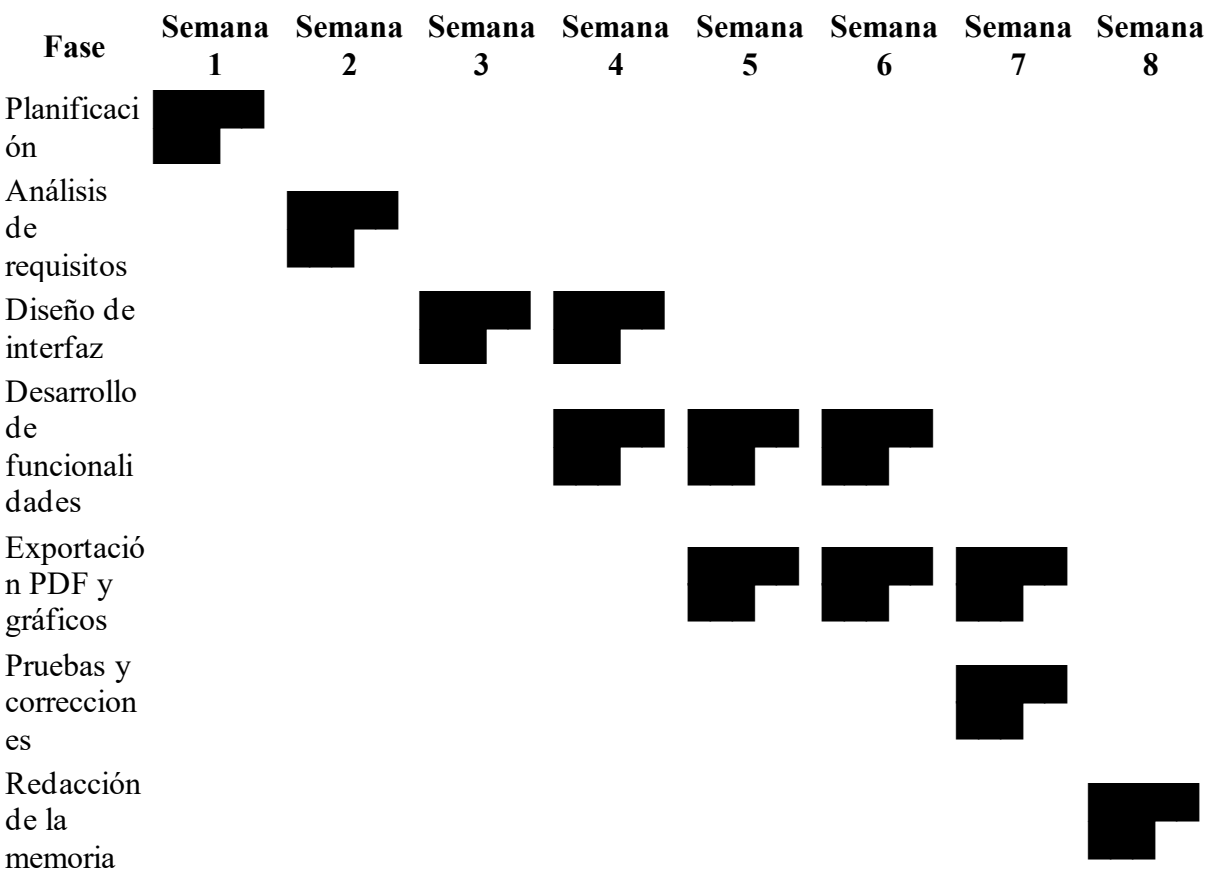
El proyecto se ha desarrollado de forma individual en un entorno académico, sin financiación externa ni equipo adicional, lo que ha condicionado tanto los tiempos de desarrollo como la elección de herramientas. A pesar de ello, se ha logrado cumplir con los objetivos principales del diseño y funcionalidad propuestos.

## 5.2. Planificación del proyecto

Para estructurar y organizar el trabajo, se dividió el desarrollo en distintas fases siguiendo una lógica cronológica. A continuación, se presenta una tabla con las fases estimadas, su duración y el tiempo real invertido:

| Fase | Duración estimada | Duración real | Descripción |
|---|---|---|---|
| **Planificación** | 4 dias | 3 dias | Definición del objetivo, herramientas, fases y tiempos |
| **Análisis de requisitos** | 4 dias | 2 dias | Identificación de funciones claves y públicos objetivos |
| **Diseño de interfaz** | 2 semanas | 1 semana | Estructura visual y navegación básica |
| **Desarrollo de funcionalidades** | 3 semanas | 4 semanas | Programación de lógica, vistas y gestión de datos |
| **Exportación PDF + gráficos** | 5 dias | 2 dias | Implementación de informes y representaciones gráficas |
| **Pruebas y correcciones** | 1 semana | 1 semana | Verificación de errores, test de usuario |
| **Documentación** | 1 semana | 2 semanas | Redacción de la memoria final del TFG |

## 5.3. Diagrama de Gantt




## 5.4. Análisis DAFO

A continuación, se presenta un análisis DAFO para evaluar el estado del proyecto desde una perspectiva estratégica:

**Debilidades**

- Desarrollo individual, sin apoyo de un equipo de diseño o testing profesional.
- Limitación de funcionalidades avanzadas por falta de tiempo y recursos.
- Sin integración con APIs bancarias reales por motivos de seguridad y privacidad.

**Amenazas:**

- Alta competencia en el mercado de apps de gestión financiera.
- Reticencia de algunos usuarios a confiar en apps nuevas sin respaldo comercial.
- Posible obsolescencia rápida si no se actualiza con nuevas funciones.

**Fortalezas**

- Aplicación sencilla, intuitiva y sin necesidad de conexión a internet.
- Totalmente gratuita, sin anuncios ni funciones bloqueadas.
- Consumo de recursos bajo y alta estabilidad en dispositivos Android.

**Oportunidades**

- Potencial publicación en tiendas oficiales (Google Play / App Store).
- Posibilidad de futuras actualizaciones para añadir sincronización en la nube.
- Creciente interés en educación financiera y autocontrol del gasto.

## 5.5. Conclusión

La organización del proyecto ha permitido cumplir los objetivos marcados dentro del marco temporal estimado. Pese a las limitaciones de recursos y tiempo, se ha logrado un producto estable, funcional y con proyección real para su uso cotidiano. El análisis DAFO también revela que el proyecto tiene una buena base para evolucionar y competir en un nicho específico de usuarios que buscan soluciones simples y eficaces.

# 6. Análisis

La aplicación cumple con una serie de requisitos tanto funcionales como no funcionales que guían su desarrollo. A continuación, se presenta el modelado previo a la implementación, incluyendo diseño de bases de datos, los diagramas de uso, clases y vistas representativas de la interfaz junto con los requisitos establecidos para su correcto desarrollo.

## 6.1. Requisitos

### 6.1.1. Requisitos funcionales

Los requisitos funcionales definen las funcionalidades específicas que debe ofrecer la aplicación. En este caso, se trata de una aplicación móvil de control de gastos personales, orientada a facilitar la gestión financiera individual mediante un entorno intuitivo, accesible y sin necesidad de conocimientos avanzados.

A continuación, se detallan los requisitos funcionales más relevantes:

- Registro de usuario:
  - El usuario podrá registrarse proporcionando un nombre de usuario, un correo electrónico y una contraseña.
  - No se solicitarán datos personales adicionales ni bancarios.
- Inicio de sesión:
  - La aplicación permitirá iniciar sesión con las credenciales registradas.
  - Se validarán los datos introducidos y se denegará el acceso si no coinciden con lo registrado.
- Visualización de categorías:
  - Dentro de cada categoría principal, hay subcategorías (por ejemplo: coche dentro de transporte, internet dentro de hogar, cocina dentro de hobbies).
  - Las subcategorías permiten una mayor personalización y detalle en el control del gasto.

- Registro de gastos:
  - El usuario podrá registrar un nuevo gasto especificando fecha, cantidad y descripción
  - El gasto queda asociado a la subcategoría especificada.
- Registro de ingresos:
  - Se podrá ingresas la nómina del usuario especificando cantidad, mes y año.
  - Esto permite llevar un seguimiento del dinero recibido cada mes.
- Visualización de movimientos:
  - La aplicación mostrara una lista con los ingresos y gastos con la siguiente información:
    - Fecha (formato (DD/MM/YYYY)
    - Cantidad
    - Descripción
- Generación de informes en PDF
  - El usuario podrá generar un informe que recopila los ingresos y gastos acumulados.
  - El informe se puede guardar en el teléfono y se puede compartir en formato PDF.
- Visualización del gráfico:
  - El gráfico se podrá visualizar en la misma pantalla de la aplicación y este se mostrará:
    - ✓ Ingresos
    - ✓ Gastos
  - Con esto, podrás hacer un balance de lo que se lleva gastado en comparación con lo ingresado.
- Cierre de sesión:
  - El usuario puede cerrar sesión en cualquier momento, simplemente saliendo de la aplicación.
  - La aplicación no mantendrá la sesión activa si se reinicia la app o se apaga el dispositivo,

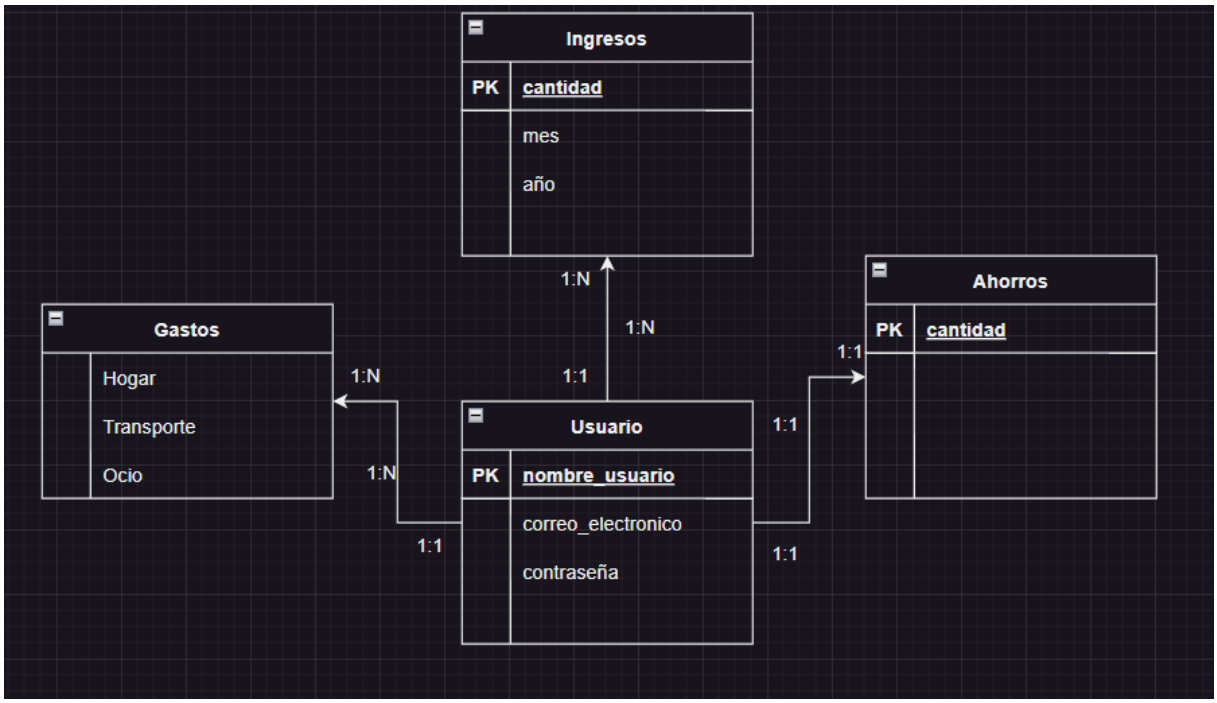
### 6.1.2.Requisitos no funcionales

Los requisitos no funcionales hacen referencia a las características de calidad que debe cumplir la aplicación, así como a sus limitaciones técnicas y de entorno. A continuación, se enumeran los requisitos no funcionales más relevantes para este proyecto:

- Usabilidad:
  - La interfaz debe ser sencilla, clara y amigable, permitiendo el uso por personas sin experiencia técnica.
  - Se utilizarán elementos gráficos y blanco y negro para facilitar la comprensión.
- Accesibilidad:
  - El diseño será adaptable a pantallas de distintos tamaños (smartphones y tablets).
  - Se evitará el uso excesivo de texto y se favorecerá el uso de iconos intuitivos.
- Estabilidad:
  - La aplicación debe funcionar si errores, bloqueos o cierres inesperados.
  - Se probará el rendimiento bajo distintos escenarios para asegurar su estabilidad.
- Rendimiento:
  - Las acciones como registrar un gasto o generar un PDF deben ejecutarse de forma rápida.
  - Se optimizarán los procesos para que la aplicación sea fluida incluso en dispositivos con recursos limitados.
- Seguridad:
  - La aplicación debe funcionar correctamente sin conexión a internet.
  - Todas las operaciones, incluidos los registros, gráficos e informes PDF, se realizarán de manera local.

- Portabilidad:
  - El código está desarrollado en Flutter para asegurar su compatibilidad con diferentes plataformas y dispositivos.
  - Aunque inicialmente se ha desarrollado para Android, la arquitectura permitiría su adaptación a IOS en el futuro.
- Mantenimiento:
  - La aplicación se ha estructurado para facilitar futuras actualizaciones y correcciones.
  - Las clases y funcionalidades están separadas de forma lógica, siguiendo buenas prácticas de programación.

## 6.2. Modelo de Datos

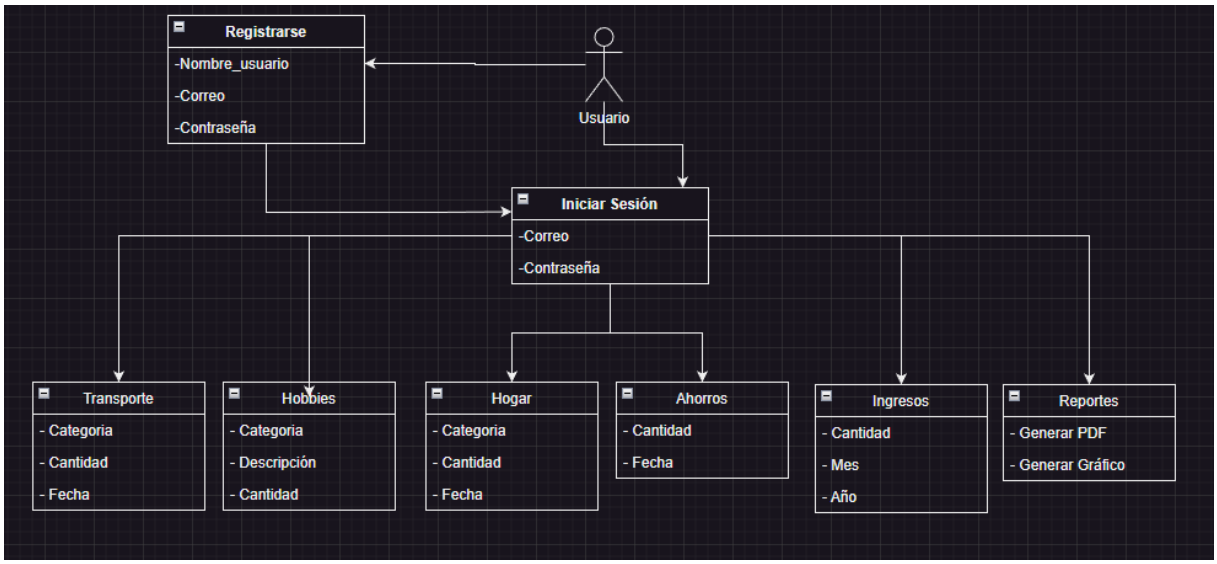



### 6.2.1. Explicación modelo de datos

- Relación usuario-ingresos: Un usuario puede tener uno o varios ingresos, pero un ingreso solo puede tener un usuario.

- Relación usuario-ahorros: Un usuario puede tener un ahorro (que realmente seria como una hucha virtual) y un ahorro solo puede tener un usuario
- Relación usuario-gastos: Un usuario puede tener uno o varios gastos mientras que un gasto solo puede tener un usuario

## 6.3. Modelo de Casos de Uso

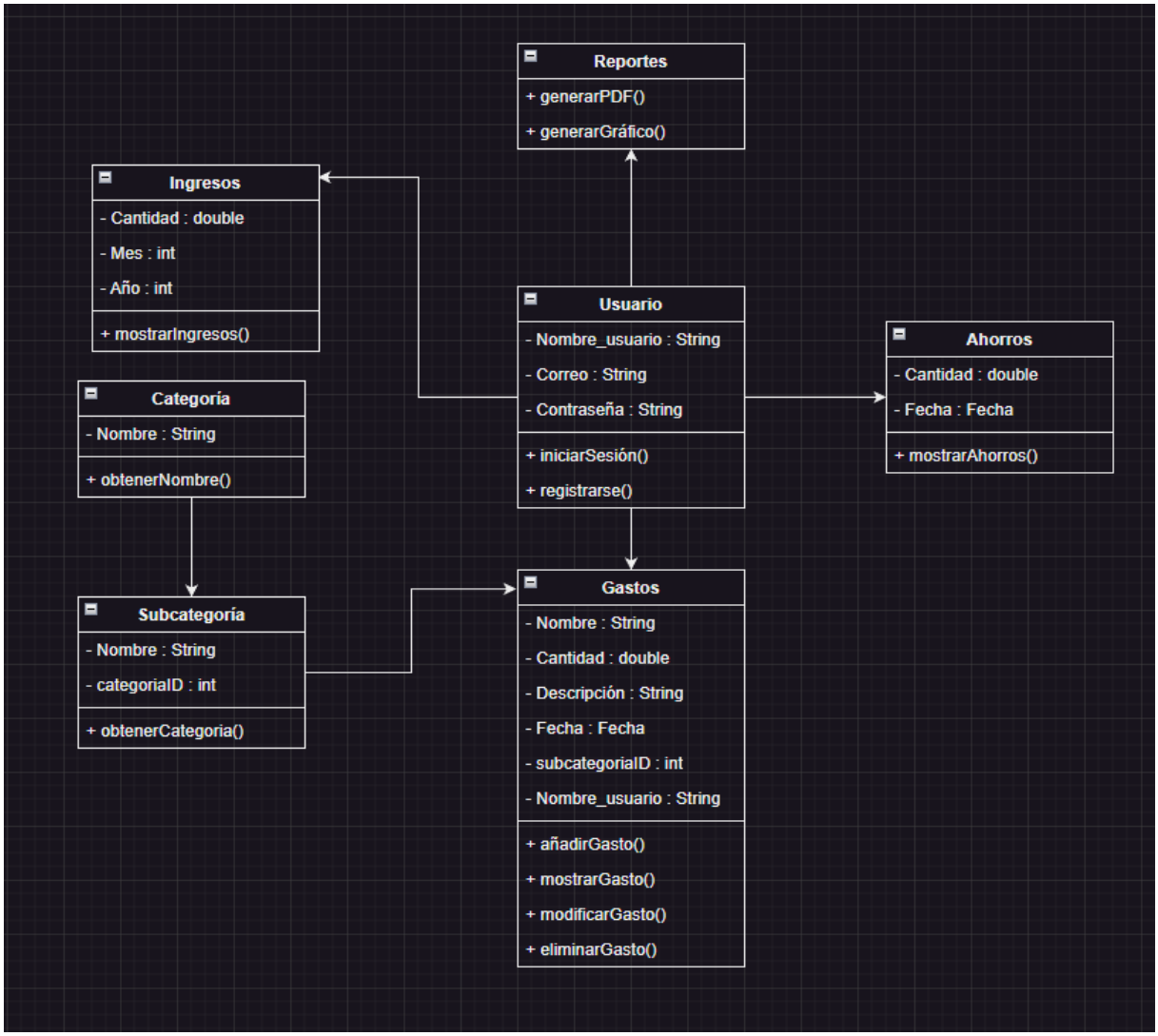



### 6.3.1. Explicación de los casos de uso

| ***Caso de uso*** | ***Descripción breve*** |
|---|---|
| *Registrarse* | Registrar credenciales (correo, contraseña y usuario) |
| *Iniciar Sesión* | Acceder con las credenciales |
| *Transporte* | Ver, agregar, modificar y eliminar gastos de coche, transporte público y viajes |
| *Hobbies* | Ver, agregar, modificar y eliminar gastos de los hobbies seleccionados |
| *Hogar* | Ver, agregar, modificar y eliminar gastos dl hogar |

| | |
|---|---|
| *Ahorros* | Agregar a la hucha virtual dinero que hayas ahorrada dicho mes |
| *Ingresos* | Agregar la nómina del mes |
| *Reportes* | Generar PDF y gráfico |

## 6.4. Modelo de Clases



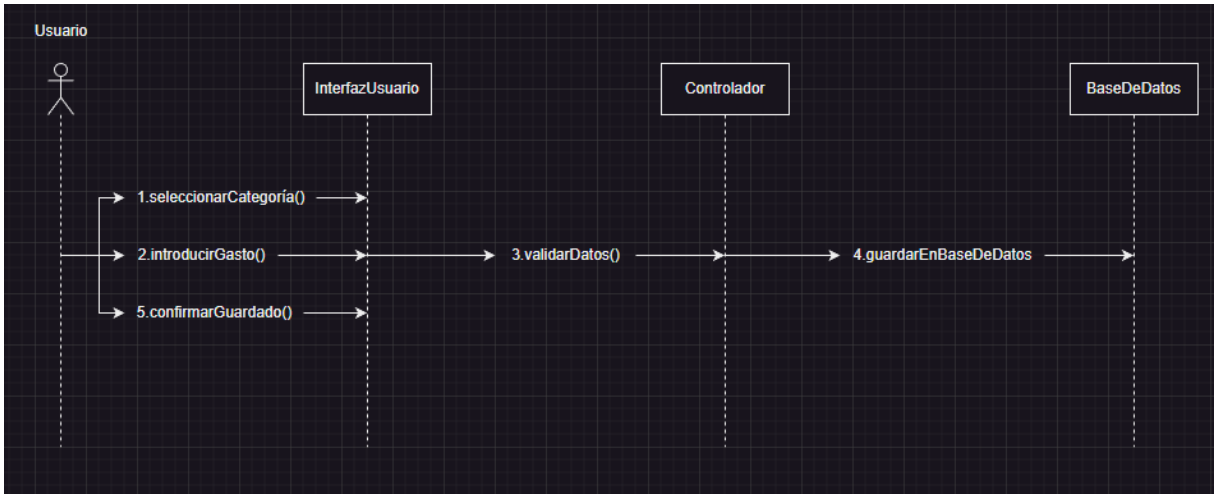
### 6.4.1. Explicación modelo de clases

- El usuario tiene:
  - Ahorros
  - Gastos
  - Ingresos
- Dentro de las categorías están las subcategorías que es donde se producen los gastos

## 6.5. Otros diagramas

En esta última parte del análisis, se presentan una serie de diagramas adicionales que permiten entender mejor el flujo, comportamiento y arquitectura del sistema.

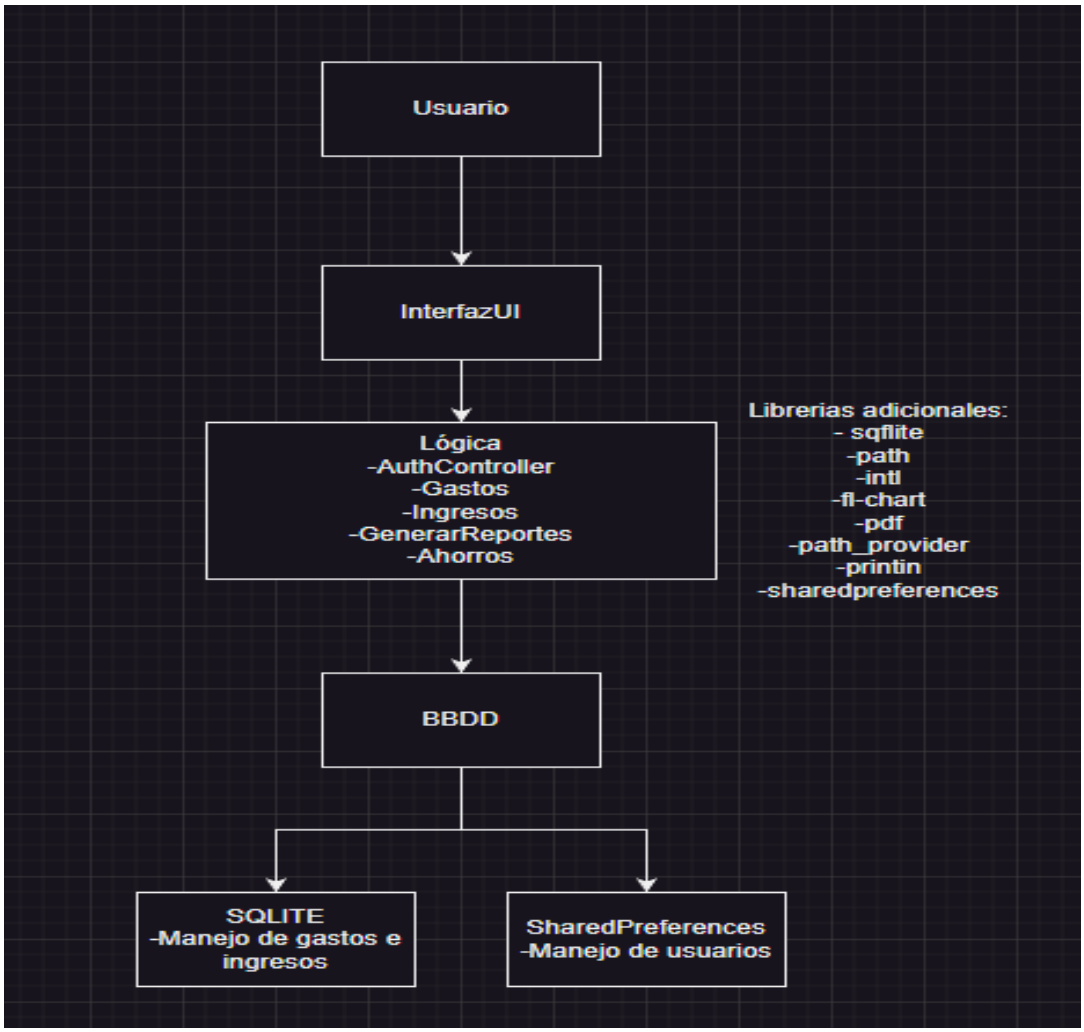
### 6.5.1.Diagrama de secuencia: Registrar un Gasto




El diagrama de secuencia muestra la interacción entre el usuario y los componentes del sistema al registrar un nuevo gasto. El usuario comienza seleccionando la categoría del gasto y completando un formulario con los datos necesarios. La interfaz transmite estos datos al controlador, que se encarga de validar la información. Si todo es correcto, el

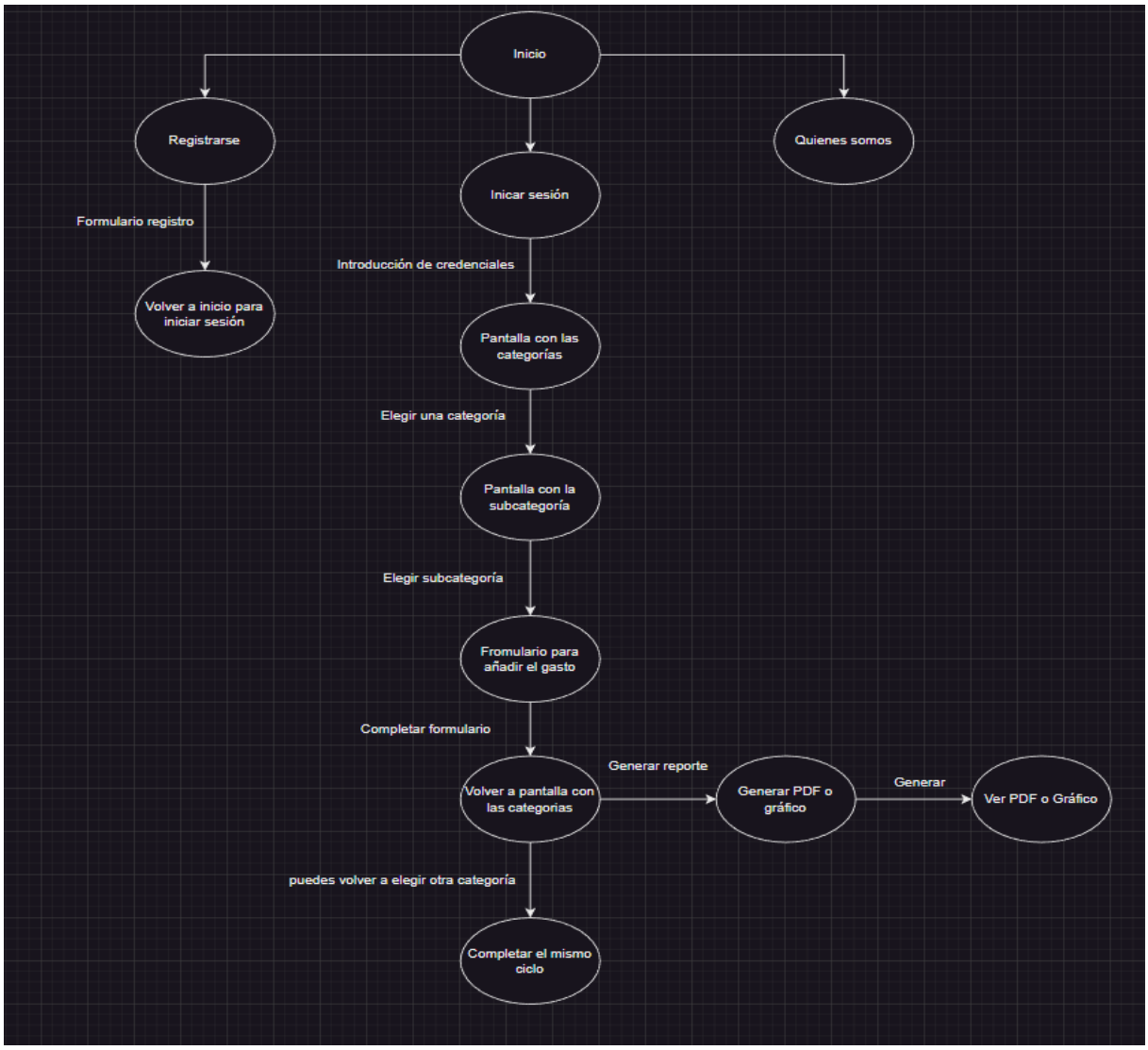
controlador envía los datos a la base de datos local (SQLite) para su almacenamiento. Finalmente, se notifica al usuario que el gasto ha sido registrado correctamente.

### 6.5.2.Diagrama de estados: Navegación de la app




Este diagrama muestra la estructura modular del sistema, detallando los principales componentes y sus interacciones. Cada componente representa una unidad funcional que colabora con las demás para cumplir con los objetivos del proyecto, facilitando así la mantenibilidad y escalabilidad del sistema.

### 6.5.3.Diagrama de componentes/arquitectura




Este diagrama representa los diferentes estados de la aplicación durante la navegación, así como las transiciones entre ellos. Cada estado corresponde a una pantalla o vista, y las transiciones muestran cómo el usuario puede moverse de una sección a otra, asegurando una experiencia fluida e intuitiva.

# 7. Diseño

## 7.1. Diseño de la interfaz

En lugar de comenzar el proyecto con wireframes o mockups detallados, se optó por un enfoque iterativo. A medida que se desarrollaban las funcionalidades, se fue ajustando el diseño de la interfaz directamente sobre el entorno de desarrollo (Android Studio con Flutter), probando distintas disposiciones y estilos visuales hasta alcanzar una versión que cumpliera con las expectativas de usabilidad, simplicidad y claridad visual.

Inicialmente, el diseño planteado era más complejo e incluía una mayor cantidad de opciones visibles desde la pantalla principal, pero tras probarlo en un entorno real, se concluyó que saturaba al usuario y no ofrecía una experiencia clara. Por ello, se rediseño la interfaz apostando por una estructura más limpia y directa, centrada en mostrar solo la información esencial: categorías de gastos, movimientos recientes y opciones de visualización general.

Este enfoque permitió centrar la atención del usuario en lo importante: **consultar, introducir y visualizar sus gastos de forma inmediata y sencilla**, sin elementos que entorpezcan la navegación. Gracias al uso de Flutter, fue posible aplicar estos cambios de forma rápida y flexible.

La interfaz se compone de las siguientes pantallas principales:

- **Pantalla de autenticación**: Permite al usuario iniciar sesión o registrarse con un correo electrónico y contraseña.

- **Pantalla de inicio**: Muestra las distintas categorías de gasto (transporte, ocio y hogar), presentadas con botones o tarjetas claras y accesibles.

- **Pantalla de registro de gasto**: Formulario para introducir el gasto (cantidad, descripción, fecha).

- **Pantalla de resumen**: Gráfica generada con la librería Fl_chart y opción para generar informe PDF.

- **Pantalla de hucha virtual**: Vista sencilla con la cantidad total disponible, basada en los ingresos y gastos introducidos.

- **Pantalla de ingresos**: Vista sencilla para que el usuario puede ver el total ingresado e ingresar por mes y año la cantidad que corresponda.

A continuación, se incluyen capturas de las principales pantallas de la aplicación, acompañadas de una breve descripción.

## 7.2. Capturas reales de la aplicación

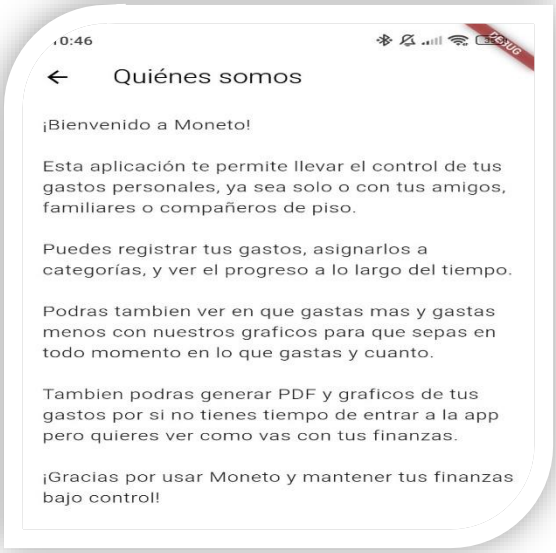
A continuación, se adjuntan una serie de capturas reales de la app, testeada en el móvil personal, enseñando lo principal y explicando en un pie de foto que parte de la app es, y que función realiza.




Pantalla principal, lo primero que se ve nada más entrar a la app, con tres botones para, registrarse, iniciar sesión, y saber un poco más de que va la app.




Página para introducir los datos del usuario




Aquí se explica un poco de que es la app y que se puede hacer.

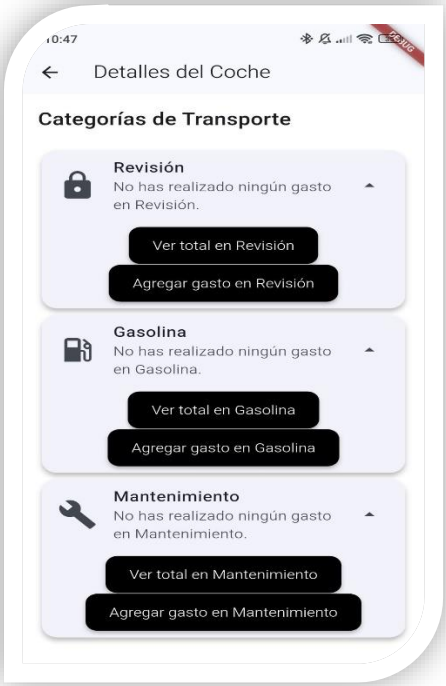



Página para introducir las credenciales que hayamos puesto en la página de registro



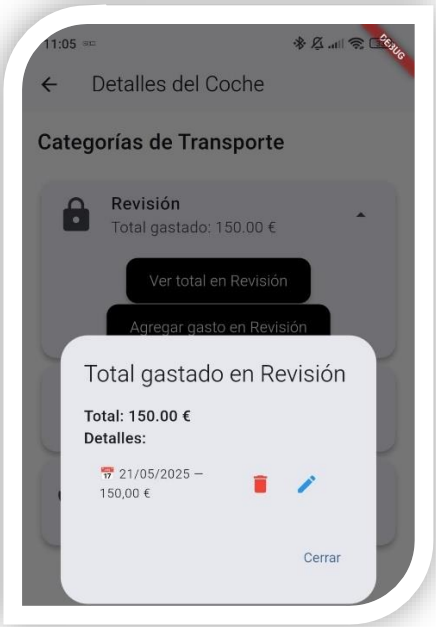

Página principal después del inicio de sesión, desde aquí se accede a todas las funciones de la app.










Aquí vemos el proceso de entrar a una de las tarjetas, en este caso, Transporte, y si entramos dentro de coche, por ejemplo, podemos ver cómo está organizado en tarjetas que se expanden, y cuando lo hacen, muestran dos botones para ver el total si hay agregado

algún gasto, y para agregar algún gasto, como podemos ver en las capturas si hay algún gasto agregado puedes o bien verlo o modificarlo o eliminarlo.





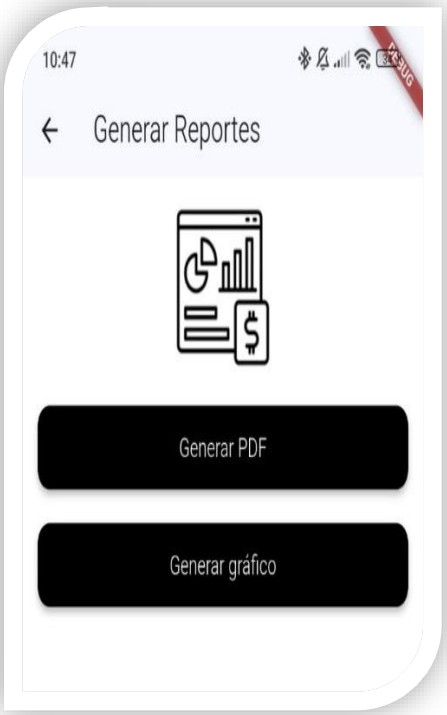


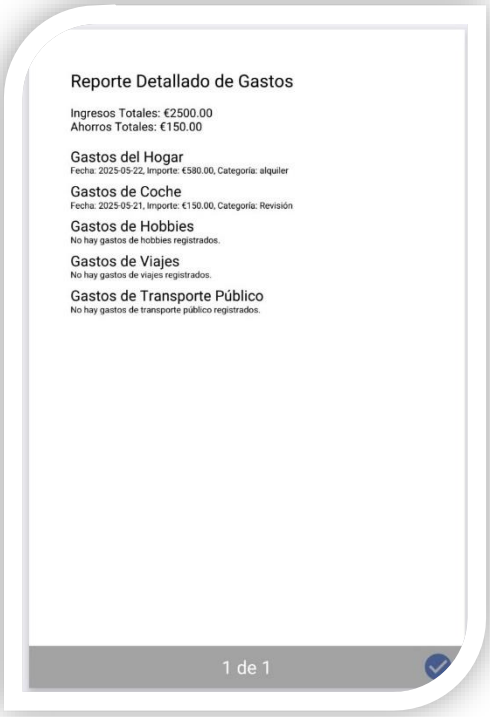






En estas capturas podemos ver las pantallas para, ver los ahorros y los ingresos, y también podemos observar cómo es para agregar dinero a la hucha o agregar el salario.



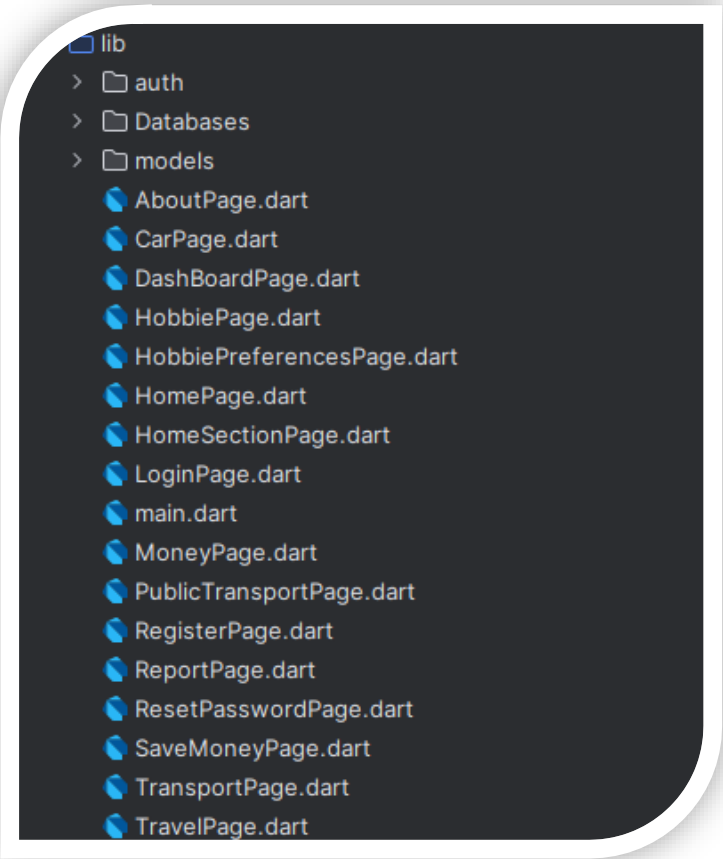




En estas tres últimas capturas, podemos ver la pantalla de reportes, como se ve el PDF generado y el grafico de gastos-ingresos.

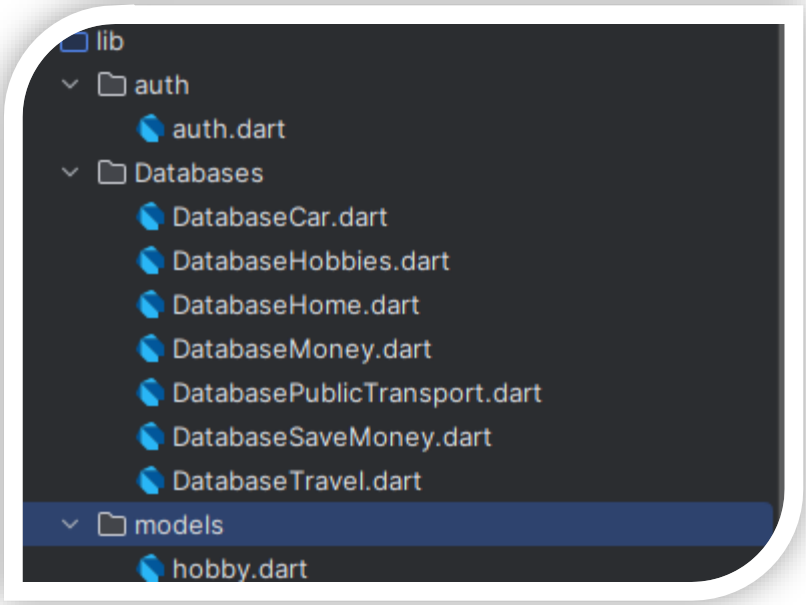
## 7.3. Arquitectura y Estructura del Proyecto

Para desarrollar el proyecto se ha optado por usar una estructura por carpetas clara en Flutter, a su misma vez haciendo el modelo MVC para el desarrollo de las bases de datos. A continuación, una explicación de las carpetas y módulos del proyecto.

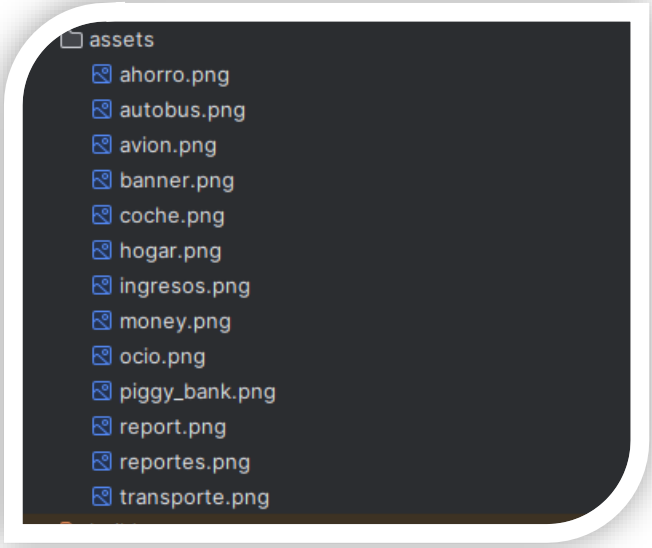



Dentro de lib, estan todas las páginas de la aplicación junto con tres carpetas, las cuales son:

- Auth: contiene la lógica para el guardado de usuarios
- Databases: Contiene los archivos para guardar los datos de cada sección.
- Models: Contiene la lógica para el guardado de las preferencias del usuario en cuanto a hobbies se refiere.




Las imágenes utilizadas se encuentran en una carpeta llamada assets.



## 7.4. Decisiones de diseño

### 7.4.1. Flutter y Android Studio

La elección de **Flutter** como framework principal para el desarrollo de la aplicación fue una decisión estratégica orientada a la **multiplataforma**. Desde el inicio del proyecto, uno de los objetivos principales era crear una aplicación funcional en distintos entornos —Android, iOS e incluso escritorio— sin necesidad de desarrollar varias versiones utilizando distintos lenguajes o tecnologías. Flutter, desarrollado por Google, permite compilar desde un único código base para todas estas plataformas, lo que facilita enormemente el mantenimiento, la escalabilidad y el despliegue futuro de la aplicación.

Además, conforme avanzaba el desarrollo, el proyecto fue adquiriendo una estructura robusta y estable. Esto abrió la posibilidad real de **publicarlo en plataformas como la Play Store o la App Store**, lo que reforzó aún más la conveniencia de utilizar Flutter, ya que garantiza compatibilidad nativa y buen rendimiento en ambas tiendas oficiales.

Como entorno de desarrollo, se seleccionó **Android Studio**, dado que ofrece una integración excelente con Flutter, además de una amplia gama de plugins y herramientas que optimizan tanto el desarrollo como la depuración de la aplicación. Su compatibilidad con Flutter SDK y su facilidad para gestionar proyectos híbridos lo convierten en una de las mejores opciones para este tipo de desarrollo.

### 7.4.2. Gestión de datos y autenticación: uso de SharedPreferences y SQLite

Para la gestión del inicio de sesión del usuario se optó por **SharedPreferences**, una solución ligera y eficiente que permite **almacenar datos clave-valor de forma persistente** en el dispositivo. Esta herramienta es ideal para guardar información básica como el correo y la contraseña del usuario (de forma segura, sin información sensible) y permitir un acceso rápido sin requerir una conexión constante a Internet. Dado que el sistema de autenticación es sencillo y no requiere validaciones en línea, **SharedPreferences cumple perfectamente con los requisitos de bajo consumo de recursos y velocidad**.

Por otro lado, para el **almacenamiento estructurado de los ingresos y gastos del usuario**, se eligió **SQLite**, una base de datos relacional embebida en el dispositivo. Esta tecnología ofrece la capacidad de realizar **consultas complejas, relaciones entre tablas y persistencia de los datos**, sin necesidad de conexión externa. Dado que la aplicación debía operar sin conexión y manejar múltiples categorías de gastos, así como fechas y descripciones asociadas, **SQLite resultó ser la opción más adecuada para garantizar un acceso rápido y estructurado a los datos**.

Ambas tecnologías permiten que la aplicación funcione **de manera completamente local, sin comprometer la privacidad del usuario ni requerir servidores externos**, lo que refuerza uno de los objetivos clave del proyecto: ofrecer una herramienta **privada, ligera y accesible para todos los públicos**.

### 7.4.3. Diseño de la interfaz y enfoque funcional simplificado

Uno de los objetivos clave del proyecto fue desarrollar una aplicación **accesible para cualquier tipo de usuario**, independientemente de su nivel de conocimientos tecnológicos. Por ello, se diseñó una **interfaz sencilla, clara e intuitiva**, donde las acciones principales (como registrar un gasto o visualizar el historial) están al alcance de pocos toques y con una estructura visual limpia. Este diseño **minimalista** evita sobrecargar al usuario con opciones innecesarias y mejora notablemente la experiencia de uso, especialmente en perfiles menos familiarizados con tecnologías móviles.

En línea con este enfoque de simplicidad, se tomó la decisión de **no implementar funcionalidades más complejas como la creación de presupuestos avanzados o gráficos interactivos excesivamente detallados**. Aunque se incluye una funcionalidad gráfica básica para ofrecer una visión general de los ingresos y gastos, se evitó incorporar herramientas de análisis financiero avanzado para mantener la aplicación enfocada en su propósito principal: **ayudar al usuario a visualizar en qué gasta su dinero y cuánto dinero entra cada mes, sin generar fricción en la experiencia**.

Este tipo de decisiones fueron clave para garantizar que la herramienta se perciba como **útil, rápida y directa**, sin requerir una curva de aprendizaje elevada ni conocimientos previos en finanzas personales.

# 8. Ejecutable y pruebas

## 8.1. Generación del ejecutable

La aplicación se ha desarrollado en Flutter, lo que permite generar ejecutables tanto para Android como para otros sistemas operativos. Sin embargo, en este proyecto se ha centrado la generación del ejecutable en la plataforma Android, utilizando el entorno de desarrollo Android Studio.

Para generar el archivo instalable (.apk), se siguió el procedimiento estándar de Flutter:

1. Abrir el proyecto en Android Studio.

2. Ejecutar el comando: flutter build apk –release

3. Esto genera el archivo APK en la carpeta /build/app/outputs/flutter-apk/app-release.apk.

El archivo .apk obtenido se instaló en un dispositivo Android real para realizar las pruebas y verificar el funcionamiento completo de la aplicación.

## 8.2. Entorno de pruebas

Las pruebas se han realizado íntegramente en un dispositivo físico: un teléfono móvil Android. El uso de un terminal real ha permitido comprobar con mayor precisión el comportamiento de la aplicación bajo condiciones reales, como el uso sin conexión a internet, cambios de orientación de pantalla, notificaciones y consumo de recursos.

**Dispositivo utilizado para las pruebas:**

- Marca: Xiaomi
- Modelo: Poco X6 pro 5G
- Versión de Android: 15 (Aunque usa su propio sistema operativo llamada Xiaomi HyperOS)
- Memoria RAM: 8 GB + 4 GB virtuales
- Espacio de almacenamiento: 256 GB
- Estado de red: pruebas con conexión WiFi y modo avión (sin conexión)

## 8.3. Proceso de pruebas

Durante el desarrollo, se ha adoptado un enfoque **incremental y validado por etapas**, es decir, cada funcionalidad se desarrollaba y se probaba de manera individual en el mismo momento. Esto permitió detectar errores de manera temprana y asegurar que cada componente funcionara correctamente antes de pasar al siguiente.

El ciclo era el siguiente:

1. Desarrollo de una pantalla o funcionalidad (por ejemplo, pantalla de ingresos).
2. Despliegue inmediato en el móvil mediante conexión USB.
3. Prueba directa de interacción, flujo, almacenamiento y navegación.
4. Corrección inmediata en caso de error.
5. Una vez verificada, se avanzaba a la siguiente pantalla.

Este enfoque ayudó a mantener un **control total del flujo de trabajo** y evitó acumulaciones de errores difíciles de rastrear.

## 8.4. Pruebas funcionales

Se realizaron pruebas de **caja negra**, centradas en verificar que cada función respondía correctamente ante una serie de entradas, sin necesidad de conocer el código fuente interno.

A continuación, se muestran algunos ejemplos de las pruebas funcionales realizadas:

| *Funcionalidad* | *Entorno de prueba* | *Resultado esperado* | *Resultado obtenido* |
|---|---|---|---|
| *Registrar ingreso* | Fecha: 05/05/2025<br>Cantidad: 2500€ | Ingreso añadido al historial correctamente | ✅ Correcto |
| *Registrar gasto en transporte > coche > revisión* | Fecha: 07/05/2025<br>Cantidad: 150€ | Gasto añadido a la categoría especificada | ✅ Correcto |
| *Generar PDF* | Generar PDF con los datos introducidos | Generación del PDF | ✅ Correcto |
| *Acceso a la app* | Usuario y contraseña registrados | Acceso a la página con todas las secciones | ✅ Correcto |
| *Acceso con datos erróneos* | Usuario o contraseña incorrectos | Mensaje de erro | ✅ Correcto |

## 8.5. Pruebas de integración

También se verificó que el sistema funcionara correctamente de forma **integrada**, comprobando cómo interactúan diferentes módulos. Por ejemplo:

- El módulo de ingreso guarda los datos en SQLite y SharedPreferences sincrónicamente.

- El módulo de generación de PDF recupera correctamente los datos almacenados y los representa en un formato legible.

- Las gráficas generadas mediante Fl_chart leen directamente los movimientos del mes seleccionado.

No se detectaron errores en las integraciones; todos los módulos interactuaban correctamente.

## 8.6. Pruebas de estabilidad y entorno

Se realizaron pruebas básicas para comprobar que:

- La app **no se cierra inesperadamente**.

- El tiempo de carga de las pantallas es **rápido** (menos de 1 segundo en navegación interna).

- La app puede funcionar sin conexión a internet.

- El consumo de recursos es **mínimo**, sin ralentizar el teléfono.

- La app se comporta correctamente tras cerrar y volver a abrir.

## 8.7. Conclusiones de las pruebas

Tras las pruebas funcionales, de integración y de estabilidad, se puede concluir que la aplicación **cumple con los requisitos establecidos** y se encuentra en un estado **funcional, estable y apto para su uso real**. No se han detectado errores críticos ni fallos de funcionamiento significativos.

# 9. Conclusiones

A lo largo del desarrollo de este Trabajo de Fin de Grado, se ha logrado cumplir con la mayoría de los objetivos inicialmente planteados, centrados en el diseño y desarrollo de una aplicación móvil destinada al control de gastos personales. Desde el inicio, el propósito principal fue ofrecer una herramienta intuitiva, clara y totalmente gratuita para facilitar a los usuarios el seguimiento de sus finanzas personales sin necesidad de conocimientos avanzados ni configuraciones complejas.

Entre los objetivos cumplidos destacan: la implementación de una interfaz funcional y sencilla que permita al usuario registrar y consultar sus ingresos y gastos clasificados por categorías; la posibilidad de generar informes en PDF con el resumen financiero mensual; y el uso de una base de datos local eficiente, basada en SQLite, que garantiza un funcionamiento fluido incluso sin conexión a Internet. Además, se logró diseñar un sistema de autenticación básico sin requerir datos personales sensibles, mejorando así la privacidad del usuario.

No obstante, algunos aspectos planteados inicialmente no se llevaron a cabo. Por ejemplo, en un principio se valoró incluir un sistema de presupuestos avanzados o gráficos más detallados. Finalmente, se optó por no implementarlos, con el fin de mantener la sencillez como una de las principales fortalezas de la aplicación. Esta decisión fue tomada tras valorar el perfil del usuario objetivo: personas que buscan una herramienta funcional, sin una curva de aprendizaje elevada ni opciones excesivamente técnicas.

Durante el proceso de desarrollo surgieron diversas dificultades. Una de las más destacadas fue la gestión del estado de la aplicación y la navegación entre pantallas en Flutter, especialmente al manejar datos persistentes. Esta dificultad se resolvió gracias a la investigación, el uso de documentación oficial y la consulta de foros especializados. Otra barrera fue mantener un diseño limpio, consistente y accesible, sin sacrificar funcionalidades, lo que implicó rediseñar varias pantallas antes de llegar a la versión final.

En el plano personal, la realización de este proyecto ha supuesto un gran aprendizaje. No solo ha permitido consolidar conocimientos técnicos en desarrollo móvil y bases de datos, sino también mejorar habilidades como la organización del trabajo, la toma de decisiones en cuanto a diseño y la gestión del tiempo en un proyecto individual a largo plazo. Además, ha reforzado la importancia de desarrollar software con el usuario final en mente, priorizando la usabilidad y la accesibilidad.

En resumen, este Trabajo de Fin de Grado ha sido una experiencia enriquecedora tanto a nivel académico como personal. El producto final es una aplicación estable, práctica y útil, que cumple con los objetivos más relevantes propuestos desde el inicio. Aunque siempre hay aspectos que podrían mejorarse o ampliarse en futuras versiones, el resultado obtenido demuestra el esfuerzo, la planificación y la dedicación invertidos durante todo el proceso.

# 10. Vías futuras

Aunque la aplicación desarrollada cumple con su función principal de gestionar gastos personales de forma sencilla, existen múltiples posibilidades de mejora y ampliación que podrían abordarse en futuras versiones. A continuación, se describen algunas ideas y líneas de evolución que podrían considerarse tanto para seguir desarrollando el proyecto como para inspirar a quienes deseen continuar su mantenimiento o expansión.

## 10.1. Implementación de gráficos más avanzados

Actualmente, la aplicación permite una visualización clara de los movimientos económicos, pero carece de gráficos visuales complejos. En futuras versiones, se podría incorporar un sistema más completo de visualización de datos mediante gráficos interactivos (circular, de barras, evolutivos por mes), que ayudarían al usuario a comprender mejor sus hábitos de gasto y a tomar decisiones más informadas sobre su economía personal. La librería Fl_chart, utilizada ya ofrece potencial para ello, solo haría falta un desarrollo más profundo y ajustado a las necesidades del usuario.

## 10.2. Sistema de presupuestos

Otra funcionalidad interesante sería la posibilidad de establecer presupuestos mensuales por categoría. De este modo, el usuario podría fijarse un límite de gasto para ocio, transporte u hogar, y la aplicación podría notificarle cuando se acerque o supere dicho umbral. Esta mejora aportaría un componente proactivo a la app, ayudando no solo a registrar lo que se gasta, sino también a controlarlo.

## 10.3. Sincronización en la nube y copias de seguridad

Actualmente, toda la información se almacena de forma local usando SQLite y SharedPreferences. Aunque esta decisión fue adecuada por motivos de simplicidad y privacidad, en el futuro podría añadirse una opción de sincronización en la nube. Esto permitiría acceder a los datos desde varios dispositivos, realizar copias de seguridad automáticas y restaurarlas en caso de pérdida o cambio de teléfono. Esta funcionalidad requeriría integrar servicios como Firebase o una API propia.

## 10.4. Multiplataforma: Versión web o de escritorio

Gracias a Flutter, el desarrollo multiplataforma es factible. Sería interesante portar la aplicación a un entorno web o de escritorio (Windows/macOS), lo que ampliaría el alcance del proyecto y permitiría a los usuarios gestionar sus finanzas desde su ordenador. Esto implicaría adaptar ciertos elementos de diseño y navegación, pero aportaría un valor añadido significativo.

## 10.5. Mejora del sistema de autenticación

Aunque el sistema actual cumple su función (basado en correo y contraseña sin validación externa), una futura mejora sería integrar un sistema de autenticación más robusto. Por ejemplo, iniciar sesión con cuentas de Google o Apple, o añadir verificación en dos pasos para aumentar la seguridad de los datos del usuario. Además, esto facilitaría

futuras integraciones con servicios externos si se conectan los datos con APIs o plataformas de terceros.

## 10.6. Sistema de exportación avanzada y análisis

La generación de informes PDF es una función útil, pero podría evolucionar para ofrecer diferentes tipos de exportación: CSV, Excel, o incluso envío automático por correo electrónico. Además, se podría integrar un pequeño sistema de análisis financiero que ofrezca recomendaciones basadas en los hábitos del usuario (por ejemplo, “Has gastado un 20% más en ocio este mes que el anterior”).

## 10.7. Interfaz personalizable

Permitir a los usuarios personalizar ciertos aspectos de la interfaz como colores, distribución de elementos o tipo de fuente podría mejorar la experiencia de uso. Esto reforzaría el enfoque de accesibilidad y adaptabilidad a distintos perfiles de usuario.

## 10.8. Internalización

Otra línea futura es adaptar la aplicación a distintos idiomas. Aunque actualmente está desarrollada en español, su estructura permite traducir textos y adaptar formatos para que sea funcional en otros mercados, lo que aumentaría el público potencial considerablemente.

## 10.9. Recomendación para futuras fases

Para poder continuar el desarrollo de esta aplicación, se recomienda mantener la filosofía de simplicidad y claridad en todo momento. Cada nueva funcionalidad debe ser evaluada no solo por su utilidad, sino también por su impacto en la usabilidad general. A medida que crecen las funcionalidades, es fundamental cuidar el rendimiento, la organización del código y la estructura modular del proyecto para facilitar su mantenimiento.

En definitiva, el proyecto tiene una base sólida y muchas posibilidades de evolución. Estas vías futuras no solo permitirían enriquecer la experiencia del usuario, sino también convertir la aplicación en una herramienta aún más potente, adaptable y útil para un público más amplio.

# 11. Bibliografía y Webgrafía

**Bibliografía**


- Sommerville, I. (2011). *Ingeniería del Software* (9.ª ed.). Pearson Educación.
- Pressman, R. S., & Maxim, B. R. (2015). *Ingeniería del software: un enfoque práctico* (8.ª ed.). McGraw-Hill.
- Freeman, E., Robson, E., Bates, B., & Sierra, K. (2014). *Head First Design Patterns: A Brain-Friendly Guide*. O'Reilly Media.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.


**Webgrafía**


- Flutter. (s.f.). *Flutter Documentation*. https://docs.flutter.dev/
- Dart. (s.f.). *Dart Language Tour* . *https://dart.dev/guides/language/language-tour*
- Sqflite. (s.f.). *sqflite | Flutter package*. https://pub.dev/packages/sqflite
- Shared Preferences. (s.f.). *shared_preferences | Flutter package*. https://pub.dev/packages/shared_preferences
- Android Developers. (s.f.). *Android Studio User Guide*. https://developer.android.com/studio

- Stack Overflow. (s.f.). *Fórum de ayuda para problemas de programación*. https://stackoverflow.com/
- OpenAI. (2024). *ChatGPT: Asistente basado en IA para redacción, código y análisis*. https://chat.openai.com/
- Google. (2024). *Gemini – Herramienta de inteligencia artificial conversacional*. https://gemini.google.com/

## 11.1. Nota sobre el uso de herramientas de inteligencia artificial:

Durante el desarrollo del proyecto, se utilizaron herramientas de inteligencia artificial como **ChatGPT** (OpenAI) y **Gemini** (Google) como apoyo en dos aspectos concretos:

- La **resolución de errores puntuales** en el código fuente cuando no se lograba detectar el fallo manualmente.

- La **redacción y mejora del estilo** en algunas secciones de la memoria, con el objetivo de expresar el contenido de forma más clara, coherente y formal.

Estas herramientas se emplearon exclusivamente como asistentes, bajo criterio propio, y en ningún caso sustituyeron el trabajo técnico ni la toma de decisiones de desarrollo realizadas por el autor del proyecto.

# Anexos

## Anexo I – Manual de Instalación de la Aplicación

---

**1. Requisitos del sistema**

Antes de instalar la aplicación, asegúrate de que tu dispositivo cumple con los siguientes requisitos:

📱 Sistema operativo: **Android 8.0 (Oreo)** o superior.

💾 Espacio disponible: **Al menos 100 MB de almacenamiento libre**.

🌐 **Conexión a internet solo necesaria durante la instalación.** Una vez instalada, la aplicación funciona sin conexión.

🔐 Permisos solicitados:

**Acceso al almacenamiento**, únicamente necesario para exportar archivos PDF generados por la app.

---

2. Instalación de la aplicación

Puedes instalar la aplicación de dos maneras:

---

✅ Opción 1: Instalación manual (archivo APK)

Esta opción es la más sencilla para usuarios que no van a usar Android Studio ni desarrollarla.

**Paso 1 – Descargar el archivo APK:**

- Solicita o descarga el archivo .apk de instalación desde una fuente segura (por ejemplo, un enlace enviado por correo o desde Google Drive).
- El archivo tiene este nombre: app-release.apk.

**Paso 2 – Transferir el archivo al móvil:**

- Puedes hacerlo de dos formas:
  - **Por cable USB:** Conecta tu móvil al ordenador y copia el archivo a una carpeta del dispositivo, como "Descargas".
  - **Por la nube:** Sube el archivo a tu Google Drive, Dropbox, etc., y descárgalo desde el móvil.

**Paso 3 – Permitir instalación desde fuentes desconocidas:**

- En tu móvil, ve a: Ajustes > Seguridad > Instalar apps desconocidas.
- Si tu móvil es reciente, debes dar permiso específico a la **app desde donde vayas a abrir el archivo** (por ejemplo, "Chrome", "Google Drive", "Gestor de archivos", etc.).
- Activa la opción: **"Permitir esta fuente"**.

**Paso 4 – Instalar la aplicación:**

- Abre el archivo app-release.apk desde el gestor de archivos.

- El sistema te pedirá confirmación para instalar. Pulsa en **"Instalar"**.
- Espera unos segundos a que termine el proceso.

**Paso 5 – Abrir y configurar la aplicación:**

- Una vez instalada, pulsa en **"Abrir"**.
- En la primera pantalla deberás:
- Registrarte con un **correo electrónico** y una **contraseña**.
- No se solicitan datos personales, bancarios ni permisos extraños.
- ¡Listo! Ya puedes comenzar a usar la aplicación.
- ---

🧑‍💻 Opción 2: Instalación desde Android Studio (modo desarrollador)

Esta opción está dirigida a usuarios con conocimientos técnicos o desarrolladores.

**Paso 1 – Requisitos técnicos:**

- Tener instalado:
- **Android Studio** (versión recomendada: Hedgehog o superior).
- **Flutter SDK** versión 3.x.
- Plugins de **Flutter**, **Dart** y **Git**.

**Paso 2 – Preparar el móvil:**

- Conectar el dispositivo Android al ordenador por USB.
- Activar las opciones de desarrollador:
- Ir a: Ajustes > Acerca del teléfono > Número de compilación y pulsar 7 veces hasta activar el modo desarrollador.
- Luego ir a: Ajustes > Opciones de desarrollador > Depuración USB y activarla.

**Paso 3 – Ejecutar la aplicación:**

- Abrir el proyecto Flutter en Android Studio.
- Con el dispositivo conectado, pulsar el botón ▶ **(Run)** para compilar y ejecutar la app.
- La aplicación se instalará directamente en el móvil, pidiendo los permisos necesarios.

---

📝 Notas adicionales:

Si al instalar ves un mensaje como “Aplicación no instalada”, asegúrate de haber descargado correctamente el archivo y que no está corrupto.

La app **no incluye publicidad, ni accede a tu ubicación ni a tus contactos.**

Anexo II – Manual de Usuario

---

**1. Pantalla de Inicio de Sesión**

Cuando abras la aplicación, verás una pantalla donde podrás iniciar sesión o registrarte.

🔐 **Iniciar sesión:**

- Introduce tu **correo electrónico** y tu **contraseña**.
- Pulsa el botón **“Iniciar sesión”**.

🆕 **Registrarse (si eres nuevo usuario):**

- Pulsa en el enlace **“Registrarse”** o en el botón correspondiente.
- Introduce:

- Un **nombre de usuario** (puede ser tu nombre o un alias).
- Un **correo electrónico válido**.
- Una **contraseña segura** (mínimo 8 caracteres recomendados, una mayúscula, una minúscula y un carácter especial).
- No es necesario confirmar el correo. Una vez completes los datos, podrás entrar directamente.

---

**2. Pantalla Principal**

Una vez dentro, accederás al menú principal donde puedes gestionar tus finanzas personales.

📊 **Categorías de Gasto:**

- Transporte
- Ocio
- Hogar

💰 **Categorías de Ingreso:**

- Ahorros
- Ingresos

📈 **Categoría de Reportes:**

Desde esta sección puedes acceder a herramientas de análisis y exportación:

- **Generar PDF**: Crea un informe con todos los movimientos registrados, agrupados por categoría y ordenados por fecha.
- **Generar Gráficos**: Visualiza tus datos en forma de gráficos circulares o de barras, que te permiten ver de forma clara cómo distribuyes tus gastos e ingresos.

📥 Todos los informes PDF se guardan automáticamente en la carpeta /Download de la memoria interna de tu dispositivo.

🔄 Estas opciones te permiten llevar un control visual y exportable de tus finanzas personales de forma rápida y sencilla.

Cada categoría representa un tipo diferente de movimiento económico. Puedes añadir entradas nuevas pulsando sobre la categoría que desees.

---

**3. Añadir un nuevo gasto o ingreso**

Después de seleccionar una categoría, verás una opción para **añadir una nueva entrada**. Se solicitarán los siguientes datos:

📅 **Fecha:** Debe escribirse en formato **dd/mm/yyyy** (por ejemplo: 12/06/2025).

💶 **Cantidad:** Monto del gasto o ingreso (puede incluir decimales).

📝 **Descripción:** Breve explicación del movimiento, como “Billete de autobús” o “Pago nómina”.

Una vez completado, pulsa **“Guardar”** para añadir el movimiento a la lista.

---

**4. Visualización y gestión de movimientos**

📂 Dentro de cada categoría puedes consultar todas las entradas registradas.

Los movimientos están **ordenados por fecha** (más recientes arriba).

Cada entrada muestra:

- La **fecha** del movimiento.
- La **descripción** que escribiste.
- La **cantidad** ingresada o gastada.

🗑️ **Eliminar un movimiento:**

Para borrar un gasto o ingreso, **mantén pulsado sobre la entrada** deseada y confirma su eliminación.

---

**5. Exportar informe en PDF**

La aplicación te permite generar un **informe completo en PDF** con todos los movimientos registrados.

📤 Para hacerlo:

- Ve al **menú principal** (icono ☰ o en la parte superior).
- Selecciona la opción **"Exportar a PDF"**.
- El sistema generará un archivo que contiene **todos los gastos e ingresos clasificados por categorías**.

📂 El archivo se guarda automáticamente en:

- Memoria interna > Download

El nombre del archivo suele seguir el formato: informe_gastos_[fecha].pdf

Puedes abrirlo con cualquier lector de PDF, compartirlo o imprimirlo si lo necesitas.

---

### ✅ Consejos de uso

- Revisa tus movimientos cada semana para llevar un buen control de tu economía.
- Usa descripciones claras para saber fácilmente en qué gastas tu dinero.
- Genera informes PDF periódicamente si necesitas rendir cuentas o llevar un archivo personal.